

Tab 1

1. Measurement Capability Verification

(Requirement 1.1 equivalent)

Requirement (generalized)

The system shall acquire at least one measurable parameter indicative of motor health.

Verification Method: Demonstration Test

Procedure

1. Install the system on the drone using the intended interface.
2. Run the motor under controlled conditions (fixed throttle, fixed duration).
3. Record the measured signal from the system.
4. Process the signal to extract at least one diagnostic feature (e.g., amplitude, frequency component, variance, etc.).
5. Store the feature value.

Pass Criteria

- A continuous signal is successfully recorded
 - At least one numerical diagnostic feature is extracted and stored
-

2. Repeatability Verification

(MOST IMPORTANT — applies to ALL concepts)

Requirement (generalized)

For repeated tests under identical conditions, diagnostic features shall remain within a defined variation limit.

Verification Method: Repeatability Test

Procedure

1. Select a known healthy motor.
2. Fix all conditions:
 - throttle setting
 - battery
 - environment
 - test duration
 - sensor placement
3. Perform at least 5 consecutive test runs.
4. Extract diagnostic features for each run.
5. Compute:
 - mean
 - standard deviation
 - percent variation

Pass Criteria

- Variation in each feature does not exceed the defined threshold (e.g., % variation or σ -based limit)

Why this works for all concepts

- Acoustic → repeatable sound spectrum
 - Electrical → repeatable waveform
 - Mechanical → repeatable vibration signature
-

3. Non-Invasive Installation Verification

(Requirement 1.4 equivalent)

Requirement (generalized)

The system shall not require permanent modification of the drone.

Verification Method: Inspection + Demonstration

Procedure

1. Document the drone condition before installation.
2. Install the system.
3. Check for:

- cutting
 - soldering
 - drilling
 - permanent fastening
- 4. Remove the system.
- 5. Compare drone condition before and after.

Pass Criteria

- No irreversible changes
 - Drone returned to original state
-

4. Baseline Calibration Verification

(Requirement 2.1 equivalent)

Requirement (generalized)

The system shall establish a reference profile for a healthy motor.

Verification Method: Calibration Test

Procedure

1. Select a healthy motor.
2. Perform at least 5 controlled test runs.
3. Extract diagnostic features for each run.
4. Compute:
 - mean
 - standard deviation
5. Store this as the baseline profile.
6. Repeat for a second motor to confirm independence.

Pass Criteria

- Baseline profile successfully created and stored
 - Different motors produce distinct baselines
-

5. Fault Detection Verification

(Requirement 2.3 equivalent — critical)

Requirement (generalized)

The system shall detect abnormal motor behavior based on deviation from baseline.

Verification Method: Threshold Validation Test

Procedure

1. Establish a baseline profile.
2. Run a test expected to be “healthy.”
3. Run a test expected to be “abnormal” (faulty or altered condition).
4. Compute deviation of features from baseline.
5. Compare to threshold (e.g., $k \cdot \sigma$).
6. Compare system output to manual calculation.

Pass Criteria

- Healthy test is not flagged
 - Abnormal test is flagged
 - System decision matches calculated threshold logic
-

6. Classification Output Verification

(Requirement 2.4 equivalent)

Requirement (generalized)

The system shall output a clear diagnostic classification.

Verification Method: Functional Test

Procedure

1. Run multiple test cases:

- normal condition
 - borderline condition
 - abnormal condition
2. Observe system output classification.

Pass Criteria

System outputs:

- PASS → within limits
 - WARNING → near limits
 - FAIL → exceeds limits
-

7. Response Time Verification

(Requirement 2.2 + 2.5 equivalent)

Requirement (generalized)

The system shall produce a diagnostic result within a specified time.

Verification Method: Timed Test

Procedure

1. Start timer at end of data acquisition.
2. Allow system to:
 - process signal
 - extract features
 - compare to baseline
 - output result
3. Stop timer when result is displayed.

Pass Criteria

- Processing time $\leq$ specified limit (e.g., 15 s)
 - Total diagnostic time $\leq$ specified limit (e.g., 5 min)
-

8. Data Logging Verification

(Requirements 4.1–4.3 equivalent)

Requirement (generalized)

The system shall store and allow retrieval of diagnostic data.

Verification Method: Data Audit

Procedure

1. Run multiple tests.
2. Save logs.
3. Verify each log includes:
 - timestamp
 - test ID
 - classification
 - raw signal data
4. Power cycle system.
5. Retrieve logs.

Pass Criteria

- All data stored completely
- Logs retrievable after restart
- No missing or corrupted data

9. Sensor Repeatability vs. Datasheet Error Verification

Requirement 1.2

Requirement (generalized)

For repeated tests under identical throttle setting and ambient temperature, run-to-run variation in each diagnostic feature across ≥ 5 consecutive runs shall not exceed the total output error cited in the sensor's manufacturer datasheet.

Verification Method: Quantitative Repeatability Test

Procedure

1. Identify the total output error figure for the sensor in use from its manufacturer datasheet (e.g., ACS712 20A: $\pm 1.5\%$ of full scale = ± 0.30 A; ICM-42688-P: noise density in $\mu\text{g}/\sqrt{\text{Hz}}$).
2. Select a known healthy motor. Fix all test conditions: throttle setting, battery voltage, ambient temperature, test duration, and sensor placement.
3. Execute ≥ 5 consecutive test runs without disturbing the setup between runs.

4. Extract the same diagnostic feature (e.g., FFT amplitude at the fundamental electrical frequency, or RMS vibration amplitude) from each run.
5. Compute the standard deviation (σ) across the ≥ 5 runs for each feature.
6. Express the run-to-run variation as a fraction of the datasheet total output error.

Pass Criteria

The standard deviation across ≥ 5 runs for each feature does not exceed the sensor's datasheet total output error.

The variation fraction (σ / datasheet error) is documented in the test log.

The result holds for all motors tested, not just one.

Note: Design-specific notes: Sound — compare σ of fundamental amplitude to microphone SNR or sensitivity tolerance. Vibration — compare σ of FFT peak amplitude to ICM-42688-P noise density (per Betaflight blackbox log at ≥ 4 kHz). Current — compare σ of fundamental frequency amplitude to ACS712 20A total output error (± 0.30 A at 20 A range).

10. Operator Complexity Verification

Requirement 1.3

Requirement (generalized)

The data acquisition procedure shall achieve a TACOM (Task Complexity) score ≤ 3.6 , corresponding to the “Nominal” complexity level defined in Jang & Park (2022, Table 8).

Verification Method: TACOM Procedural Analysis

Procedure

1. Write out the complete step-by-step data acquisition procedure for the design under evaluation, from the operator's perspective (e.g., connect cable, open software, set throttle, trigger acquisition, retrieve result).
2. For each step, apply the TACOM scoring rubric from Jang & Park (2022), Table 8, assigning a complexity score based on: number of information items to process, number of decisions required, and simultaneity of actions.
3. Sum all step scores to obtain the total TACOM score for the procedure.
4. Compare the total to the 3.6 threshold. If any step triggers branching (e.g., “if LED is off, check connection”), count each branch as a separate step.
5. Repeat the analysis for each of the three design concepts to enable direct comparison.

Pass Criteria

Total TACOM score for the data acquisition procedure is ≤ 3.6 .

All individual step scores and their sum are documented.

The procedure as scored matches the procedure as actually performed — no undocumented steps.

Note: Source: Jang & Park (2022). A task complexity metric for evaluating human error probability in nuclear power plants. Nuclear Engineering and Technology, 54, 4170–4180, Table 8. The TACOM method was originally developed for nuclear control rooms but has been adapted to general procedural tasks. Apply consistently across all three design concepts for a valid comparison.

11. Network Independence Verification

Requirement 1.5

Requirement (generalized)

The system shall make zero external network calls during a complete diagnostic cycle.

Verification Method: Network Traffic Inspection

Procedure

1. Disconnect the test device from all Wi-Fi networks and disable mobile data (put in airplane mode or disconnect router).
2. Run a complete diagnostic cycle from initiation to result output.
3. Monitor network traffic using a packet capture tool (e.g., Wireshark on the connected laptop, or monitor the router's connected-device log).
4. Alternatively, check the system's source code and firmware to confirm no network call is issued at any stage of the pipeline.
5. Repeat with Wi-Fi re-enabled to confirm the system also functions correctly when connectivity is available.

Pass Criteria

Zero outbound network packets are observed during the diagnostic cycle.

The complete diagnostic cycle executes successfully with no internet connection.

No API calls, DNS lookups, or cloud service calls are made at any stage.

Note: Justification: Danny Nguyen confirmed regular deployment in remote areas (RFP Appendix A, §2.4). LTE coverage reaches only 96.3% of rural Canadian households and high-speed internet availability drops to 59.5% (OAG, 2023; ISED, 2019). Any network dependency is a single point of failure that could prevent the system from being used at a shoot location.

12. Operator Output Clarity Verification

Requirement 3.1

Requirement (generalized)

The system shall communicate the diagnostic classification result in a form requiring no interpretation of raw data by the operator.

Verification Method: User Comprehension Test

Procedure

1. Run the system on three motors: one healthy, one borderline, one with a known fault condition.
2. Present the system's output display (screen, LED indicator, printed report, etc.) to a person unfamiliar with the system's internal diagnostics.
3. Ask the observer: "Based on what you see, what action would you take for each motor?"
4. Record whether the observer correctly identifies: (a) which motor to pass, (b) which to monitor, (c) which to ground.
5. Separately verify that raw diagnostic data (e.g., FFT amplitudes, feature vectors) is not visible in the primary operator-facing output — it may exist in the log (Requirement 4.1) but not the operator display.

Pass Criteria

The observer correctly identifies the appropriate action for each motor without being told how the system works.

No raw numerical diagnostic data is present in the primary operator-facing output.

The output contains at most one of: PASS, WARNING, or FAIL per motor.

Note: Requirement rationale: Excessive information increases decision time and the likelihood of misinterpretation (RFP Appendix A, §1.6). The operator's role is to act on the result, not interpret sensor data.

13. Operating Temperature Verification

Requirement 5.1

Requirement (generalized)

The system shall operate in ambient temperatures between -10°C and 40°C .

Verification Method: Datasheet Review + Functional Test at Bounds

Procedure

1. Collect the manufacturer's datasheet for each hardware component in the diagnostic system (sensor, microcontroller, connectors, display if any).
2. Identify the operating temperature range stated for each component.
3. Confirm that every component's stated range encompasses -10°C to 40°C .
4. If datasheet review passes, perform a functional test at the lower bound: store the assembled system in a freezer or cold environment at approximately -10°C for 30 minutes, then run a complete diagnostic cycle immediately upon removal.
5. Perform a functional test at the upper bound: operate the system in a warm environment at approximately 40°C and run a complete diagnostic cycle.

Pass Criteria

All component datasheets confirm operating range encompasses -10°C to 40°C .

System completes a full diagnostic cycle correctly at -10°C .

System completes a full diagnostic cycle correctly at 40°C .

No sensor output degradation, connection failure, or software error occurs at either temperature bound.

Note: Known status by design: Vibration (ICM-42688-P: -40°C to $+85^{\circ}\text{C}$ — passes by datasheet). Current (ACS712: -40°C to $+150^{\circ}\text{C}$; ESP32 WROOM-32: -40°C to $+85^{\circ}\text{C}$ — passes by datasheet). Sound — verify specific microphone module datasheet; many MEMS mics are rated 0°C to 70°C and may fail the -10°C lower bound.

14. System Volume Verification

Requirement 5.2

Requirement (generalized)

The system shall fit within a volume of $8,000\text{ cm}^3$.

Verification Method: Physical Measurement

Procedure

1. Assemble the complete diagnostic system in its field-deployed configuration (all components that must travel to the shoot location with the drone).
2. Measure the external dimensions of the assembled system or its carrying case/enclosure: length (L), width (W), height (H) in centimetres.
3. Compute volume: $V = L \times W \times H$.
4. If the system has an irregular shape, compute the volume of the smallest rectangular bounding box that fully encloses it.
5. Do not include components that are already part of the drone itself (e.g., the flight controller) or standard laptop/tablet used for other purposes.

Pass Criteria

Computed volume $V \leq 8,000 \text{ cm}^3$.

Measurement and calculation are documented with recorded dimensions.

Motor Current Verification

Here is the full verification plan tied directly to your requirements, split across the three tools we've discussed.

Full Verification Plan: Motor Current (MCSA) Concept

The plan has three stages that address different layers of the design. Each stage maps to specific requirements.

Stage 1 — Sensor Feasibility (LTspice Simulation)

What requirement this addresses: Req. 1.2 (run-to-run variation $\leq$ sensor datasheet error)

What you're actually asking: Can any current sensor physically resolve the fault-induced current change against its own noise floor on this specific motor?

How to do it in LTspice:

- Model the motor phase current as a current source outputting a 1,550 Hz sine wave at your operating amplitude (~10A RMS at 20% throttle)
- Add a small sideband at the fault frequency (e.g. bearing fault frequency, calculable from the motor's 9N12P geometry) at ~1–5% of the fundamental amplitude (~0.1–0.5A) — this represents the fault signal you're trying to detect
- Model the ACS712-20A as a voltage-output gain block: 100mV/A sensitivity, $\pm 1.5\%$ full-scale noise (~ $\pm 300\text{mA}$ equivalent)
- Read the output and check: is the fault sideband amplitude larger than the noise floor?

Expected result: The ACS712 will likely fail this check — $\pm 300\text{mA}$ noise vs. a ~0.14–0.7A fault signal is marginal to insufficient. This is your evidence to reject it.

Sensor iteration: Swap the sensor model and re-run for alternatives:

- INA240A1 — high-side amplifier, much lower noise, datasheet-specified
- INA180A4 — 200V/V gain variant, very low noise floor
- Record pass/fail for each against Req. 1.2

What you present: A table of sensors tested, their SNR against the fault signal, and a conclusion — either "sensor X passes, concept is feasible" or "no sensor we found passes, concept fails Req. 1.2." Either outcome is a valid, evidence-based engineering conclusion.

Justification for LTspice: Industry-standard SPICE simulator, used in professional circuit design and ECE programs globally. Component models sourced directly from manufacturer datasheets. The underlying math (nodal analysis, noise modelling) is independently verifiable.

Stage 2 — Signal Processing Pipeline (ECE Lab, Waveform Generator)

What requirements this addresses: Req. 2.2 (diagnostic result within 15 seconds) and the core detection logic behind Req. 2.1

What you're actually asking: Does the FFT + classification algorithm correctly distinguish a healthy signal from a faulty one, and does it do so within the time constraint?

How to do it:

- Program your ESP32 (or laptop) with the FFT + classification code
- Use the waveform generator to output a voltage signal into the ADC pin where the sensor would normally connect — scaled to match what the sensor would output (e.g. if sensor outputs 100mV/A and the motor runs at 10A, you output a 1V sine wave at 1,550 Hz)
- Run Test A: healthy signal (clean 1,550 Hz sine) → algorithm should output "healthy"
- Run Test B: faulty signal (1,550 Hz + small sideband at fault frequency) → algorithm should output "fault detected"
- Time the output from signal start to classification result — verify it's under 15 seconds (Req. 2.2)
- Repeat both ≥ 5 times to assess repeatability consistency (links to Req. 2.1 logic)

What you present: Oscilloscope screenshots or serial monitor logs showing correct classification on both signal types, and timestamps confirming the 15-second requirement is met.

Honest limitation to state: The waveform generator produces a clean, noise-free signal. Real motor current has ESC switching noise (~20–48 kHz), electromagnetic interference, and load variation. The algorithm may perform worse in real conditions — this is a residual uncertainty you carry forward.

Stage 3 — Analytical Calculations (Hand / Spreadsheet)

What requirements this addresses: Req. 1.1 (at least one measurable parameter indicative of motor health) and supporting justification for Req. 1.3 ($TACOM \leq 3.6$)

What you're doing:

- Calculate the expected fault frequencies for the GEPRC motor from first principles (bearing fault frequency formula using inner/outer race geometry, number of rolling elements, contact angle) — this confirms that MCSA is theoretically capable of detecting the specific faults relevant to this motor
- Estimate TACOM score for the data acquisition procedure (how many steps does the operator need to run the test?) — compare against the ≤ 3.6 threshold

These aren't glamorous but they show you've thought through the design at a detailed level and they provide independent grounding for the simulation assumptions.

How the Three Stages Connect

Stage 1 (LTspice) Stage 2 (Waveform Generator) Stage 3 (Calculations)

"Can the sensor see the fault signal?"	→	"Does the algorithm work on a clean version of that signal?"	→	"Are the fault frequencies real? Is the procedure simple enough for a pilot?"
Req. 1.2		Req. 2.2, 2.1 (partial)		Req. 1.1, 1.3

The honest framing for your assessors is: **Stage 1 is the critical gate.** If no sensor passes the SNR check, the concept cannot meet Req. 1.2 and you have evidence-based grounds to eliminate or deprioritize it. If a sensor does pass, Stages 2 and 3 give you confidence the rest of the pipeline works, and the remaining uncertainty is real-world noise — a known, bounded risk rather than an unknown.

What This Looks Like at Beta Release

You'd present this as a **two-requirement verification plan** (satisfying the Beta Release constraint of ≥ 2 key requirements):

- **Req. 1.2 verified via:** LTspice sensor SNR simulation with ACS712 and alternatives
- **Req. 2.2 verified via:** Waveform generator + FFT pipeline test, timed

Plus supporting analytical work on fault frequencies and TACOM as background evidence. That's a complete, multi-modal, honestly-limited verification argument — exactly what the rubric describes as "Good" to "Outstanding."